

TRABAJO PRÁCTICO INTEGRADOR

Materia: Organización Empresarial

Alumno: Rodrigo Moyano

Caso de estudio: Burger Home

Repositorio: https://github.com/ElRodriCAT/TPI1_OE.git

INTRODUCCIÓN

El presente trabajo tiene como objetivo analizar y modelar el proceso de gestión de pedidos de una hamburguesería ficticia denominada Burger Home. A partir del análisis del proceso actual (AS-IS), se identifican inefficiencias relacionadas con la atención manual de clientes, el control de stock y la gestión de pedidos. Como propuesta de mejora (TO-BE), se plantea la automatización parcial del proceso mediante un chatbot simulado en Python. La solución utiliza BPMN 2.0 para representar el proceso de negocio y una Máquina de Estados Finitos para controlar las distintas etapas de la interacción, contemplando tanto el flujo principal como situaciones alternativas y errores de entrada.

DESCRIPCIÓN DE LA ORGANIZACIÓN

Nombre: Burger Home.

Rubro: Venta de hamburguesas artesanales mediante atención en local y servicio de reparto.

Tamaño: Pequeña empresa local.

Forma de trabajo actual: los pedidos se reciben principalmente por WhatsApp. Un empleado atiende manualmente las consultas, verifica disponibilidad de productos, registra los pedidos y coordina su preparación y entrega.

Problemas detectados:

- Demoras en la atención durante horarios pico.
- Errores en la toma de pedidos.
- Tiempo destinado a responder consultas repetitivas.
- Dificultad para controlar el stock de forma inmediata.
- Dependencia de la disponibilidad de un empleado para responder mensajes.

ENFOQUE Y ANÁLISIS SISTÉMICO DE BURGER HOME

Burger Home puede analizarse desde la Teoría General de Sistemas como un sistema abierto, artificial y social. Está formado por subsistemas interrelacionados -Atención/Administración, Cocina y Reparto- que intercambian información y recursos para transformar las solicitudes de los clientes en pedidos preparados y entregados.

- Entradas: materias primas, dinero y datos del cliente, pedido, dirección y comprobante de pago.
- Procesos: recepción del pedido, validación de producto y stock, verificación de cobertura, cálculo del total, preparación y entrega.
- Salidas: pedido preparado y entregado, confirmaciones al cliente y registros de pedidos realizados.
- Retroalimentación: avisos de errores, falta de stock, problemas de cobertura y reintentos que permiten corregir el flujo antes de continuar.
- Ambiente: clientes, proveedores, competidores, normativa aplicable y servicios externos de comunicación.
- Límites: personal, proceso interno de preparación y reparto, simulador en Python y mecanismo de persistencia utilizado por la solución.

En el proceso manual, la información depende en gran medida de la interpretación y disponibilidad del empleado, lo que puede producir demoras, errores y cuellos de botella. La propuesta automatizada estandariza las entradas mediante estados y validaciones, consulta reglas de negocio como stock y cobertura y genera retroalimentación inmediata cuando aparece una excepción. De esta manera se mejora la consistencia del flujo de información entre Atención, Cocina y Reparto.

ANÁLISIS DEL PROCESO ACTUAL (AS-IS)

Actualmente Burger Home recibe pedidos mediante WhatsApp. El proceso comienza cuando el cliente consulta por los productos disponibles. Un empleado comparte el menú, registra la selección y verifica

manualmente el stock. Luego consulta si el pedido será retirado en el local o enviado por delivery; en este último caso también verifica la zona de cobertura. Posteriormente calcula el importe, consulta el medio de pago y, si corresponde, solicita y verifica el comprobante de transferencia. Finalmente confirma el pedido, lo comunica a Cocina y coordina su retiro o entrega. La intervención manual en gran parte del proceso explica las demoras y errores identificados.

PROPUESTA DE MEJORA (TO-BE)

La propuesta TO-BE automatiza la recepción y validación del pedido mediante un chatbot. El sistema muestra el menú, valida el producto y el stock, solicita la modalidad de entrega, verifica cobertura cuando corresponde y gestiona el medio de pago. Ante situaciones recuperables, el proceso no finaliza innecesariamente: un producto inválido o sin stock permite volver a seleccionar; una dirección fuera de cobertura ofrece la alternativa de retiro en el local; y un comprobante inválido puede volver a ingresarse. Una vez confirmado el pedido, se registra la información, se actualiza el stock, se notifica a Cocina y el flujo continúa hasta la entrega o retiro.

ARQUITECTURA Y PROGRAMACIÓN (EL "CÓMO")

La solución se implementa como una simulación de chatbot por consola utilizando Python. El archivo `base_datos.json` funciona como mecanismo de persistencia simulado para almacenar el catálogo, el stock, las zonas de cobertura y el historial de pedidos. La lógica del programa mantiene coherencia con las decisiones representadas en el BPMN mediante estructuras condicionales y una Máquina de Estados Finitos.

Estados principales de la simulación:

- **IDLE**: espera el inicio de la conversación.
- **ESPERANDO_PRODUCTO**: solicita y valida la selección del producto.
- **ESPERANDO_MODALIDAD**: espera Delivery o Retiro.
- **ESPERANDO_DIRECCION**: solicita la dirección para Delivery.
- **ESPERANDO_CONFIRMACION_RETIRO**: ofrece Retiro cuando no existe cobertura.
- **ESPERANDO_PAGO** y **ESPERANDO_COMPROBANTE**: gestionan el medio de pago y su validación.
- **PEDIDO_CONFIRMADO**, **EN_PRODUCCION**, **LISTO** y **EN_REPARTO**: representan el avance operativo posterior a la confirmación.
- **ENTREGADO**: registra la finalización y permite volver al estado inicial.

La Máquina de Estados permite que cada entrada sea interpretada según el punto del proceso en el que se encuentra el usuario. Los errores recuperables mantienen o redirigen el estado correspondiente, evitando que una entrada incorrecta obligue a reiniciar todo el pedido.

PERSISTENCIA Y DICCIONARIO DE DATOS

La persistencia se concentra en el archivo `base_datos.json`. No se trata de una base de datos relacional, sino de un archivo JSON utilizado para simular el almacenamiento requerido por el proceso. El programa consulta y modifica sus datos durante la ejecución.

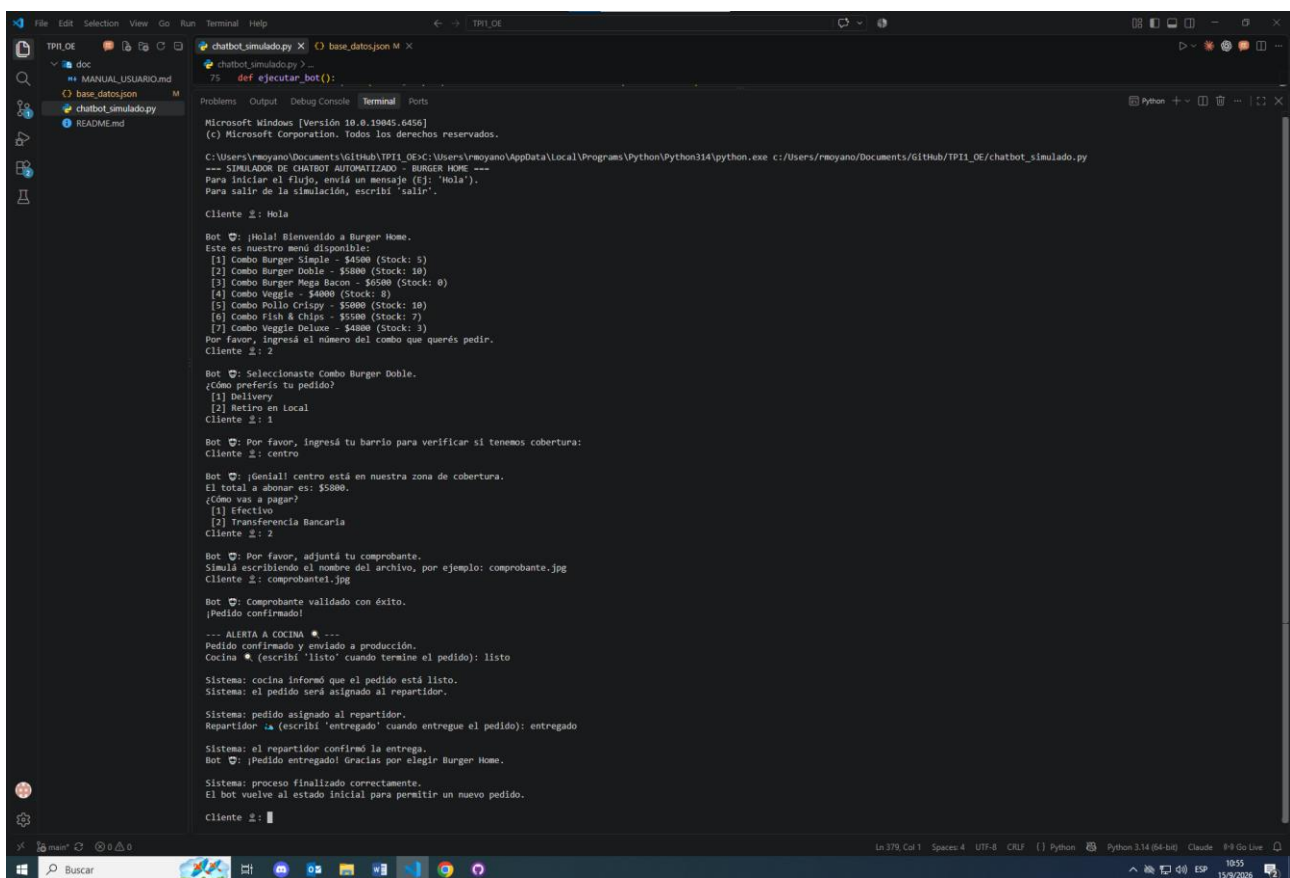
Dato	Tipo	Uso	Regla principal
estado_actual	Texto/Enum	Estado activo de la FSM	Solo admite estados definidos por el flujo
productos	Objeto JSON	Catálogo de productos	Identificados por una clave
nombre	String	Nombre del producto	No nulo
precio	Númeroico	Cálculo del total	Mayor que cero
stock	Entero	Disponibilidad	Mayor o igual a cero
zonas_cobertura	Lista	Barrios habilitados	Se compara con la entrada normalizada
pedidos_registrados	Lista de objetos	Historial persistente	Se agrega al confirmar el pedido
modalidad	String	Forma de entrega	Delivery o Retiro
direccion	String/Null	Destino del delivery	Null cuando es Retiro

ROBUSTEZ Y PRUEBAS DE POSIBILIDAD DE ESTRÉS

Se realizaron pruebas orientadas a comprobar el comportamiento del chatbot frente a entradas inválidas y caminos alternativos. Estas pruebas permiten verificar que la simulación no contemple únicamente el camino feliz, sino también situaciones frecuentes durante la interacción con un usuario.

- Producto inexistente: se informa el error y se vuelve a solicitar un producto.
- Producto sin stock: se informa la falta de disponibilidad y se permite seleccionar otro producto.
- Dirección fuera de cobertura: se ofrece Retiro en local; si el cliente acepta, el flujo continúa hacia el pago; si rechaza, el pedido se cancela.
- Método de pago inválido: se informa el error y se mantiene la solicitud de pago.
- Comprobante inválido: se informa el problema y se permite volver a ingresarlo sin reiniciar el pedido.
- Flujo posterior a la confirmación: Cocina informa cuando el pedido está listo y el proceso continúa por Retiro o Delivery hasta marcarlo como entregado.

Camino feliz de un pedido confirmado:



```
def ejecutar_bot():
    Microsoft Windows [Versión 10.0.19045.6456]
    (c) Microsoft Corporation. Todos los derechos reservados.

    C:\Users\rmyano\Documents\GitHub\TP1_0E>C:\Users\rmyano\AppData\Local\Programs\Python\Python314\python.exe c:/Users/rmyano/Documents/GitHub/TP1_0E/chatbot_simulado.py
    --- SIMULADOR DE CHATBOT AUTOMATIZADO - BURGER HOME ---
    Para iniciar el flujo, envía un mensaje (Ej: 'Hola').
    Para salir de la simulación, escribi 'salir'.

    Cliente 2: Hola

    Bot > ¡Hola! Bienvenido a Burger Home.
    Este es nuestro menú disponible:
    [1] Combo Burger Simple - $4500 (Stock: 5)
    [2] Combo Burger Doble - $5800 (Stock: 10)
    [3] Combo Burger Mega Bacon - $6500 (Stock: 0)
    [4] Combo Veggie - $4000 (Stock: 8)
    [5] Combo Pollo Crispy - $5000 (Stock: 10)
    [6] Combo Fish & Chips - $5500 (Stock: 2)
    [7] Combo Veggie Deluxe - $4800 (Stock: 3)
    Por favor, ingresá el número del combo que querés pedir.
    Cliente 2: 2

    Bot > Seleccionaste Combo Burger Doble.
    ¿Cómo preferís tu pedido?
    [1] Delivery
    [2] Retiro en Local
    Cliente 2: 1

    Bot > Por favor, ingresá tu barrio para verificar si tenemos cobertura:
    Cliente 2: centro

    Bot > ¡Genial! centro está en nuestra zona de cobertura.
    El total a abonar es: $5800.
    ¿Cómo vas a pagar?
    [1] Efectivo
    [2] Transferencia Bancaria
    Cliente 2: 2

    Bot > Por favor, adjuntá tu comprobante.
    Simulá escribiendo el nombre del archivo, por ejemplo: comprobante.jpg
    Cliente 2: comprobante1.jpg

    Bot > Comprobante validado con éxito.
    ¡Pedido confirmado!

    --- ALERTA A COCINA ---
    Pedido confirmado y enviado a producción.
    Cocina > (escribi 'listo' cuando termine el pedido): listo

    Sistema: cocina informó que el pedido está listo.
    Sistema: el pedido será asignado al repartidor.

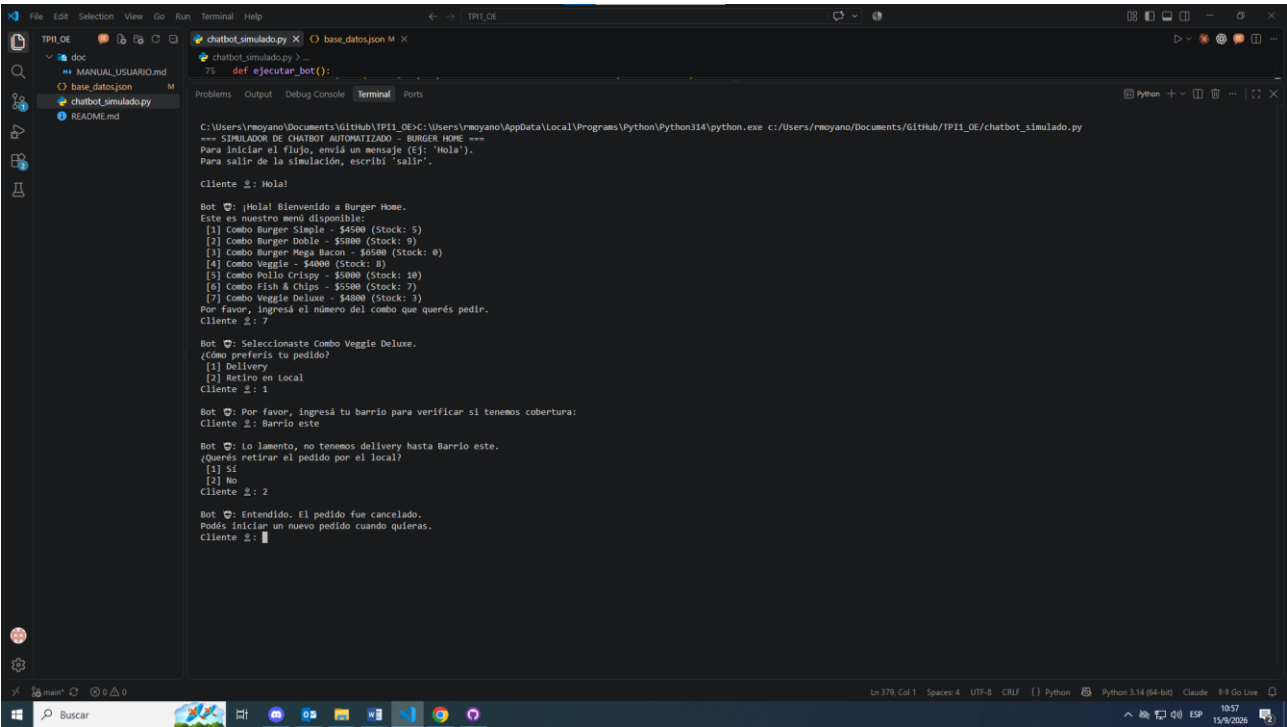
    Sistema: pedido asignado al repartidor.
    Repartidor > (escribi 'entregado' cuando entregue el pedido): entregado

    Sistema: el repartidor confirmó la entrega.
    Bot > ¡Pedido entregado! Gracias por elegir Burger Home.

    Sistema: proceso finalizado correctamente.
    El bot vuelve al estado inicial para permitir un nuevo pedido.

    Cliente 2: 
```

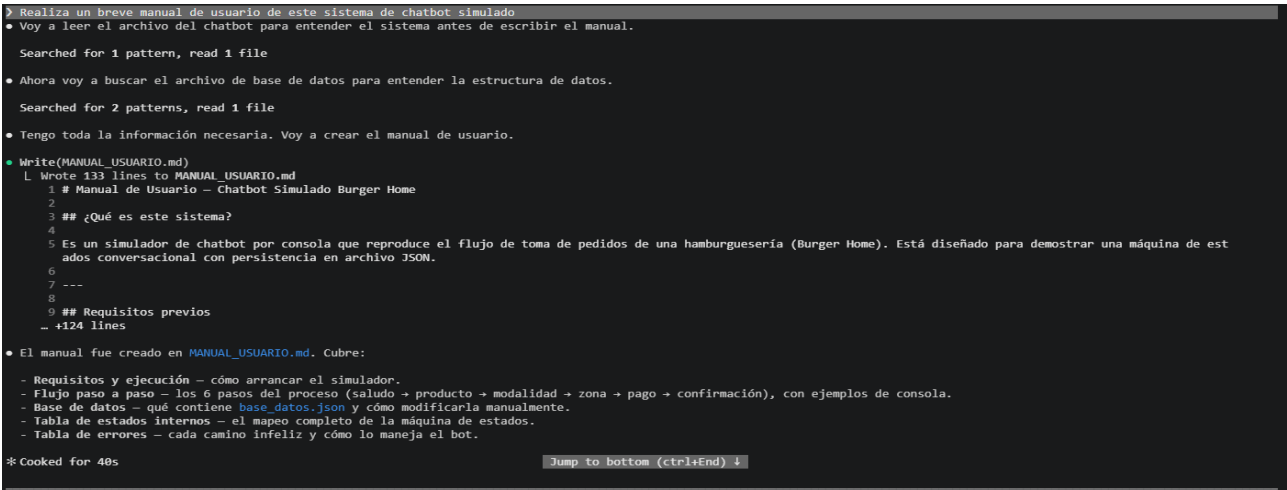
Camino infeliz de un pedido sin cobertura de delivery



HERRAMIENTAS DE INTELIGENCIA ARTIFICIAL

Durante el desarrollo se utilizaron herramientas de inteligencia artificial como apoyo para la planificación, revisión y documentación. Claude CLI se empleó para revisar el código y apoyar la generación del manual de usuario, mientras que Gemini se utilizó como herramienta de consulta durante la planificación y revisión de aspectos del modelado. Las respuestas obtenidas no se tomaron como resultado final de manera automática: fueron contrastadas con la consigna, el funcionamiento del programa y el modelo BPMN, realizando correcciones cuando se detectaron inconsistencias.

Herramienta / consulta	Objetivo	Análisis propio	Decisión aplicada
Claude CLI - revisión de código	Detectar errores e inconsistencias	Se verificaron las sugerencias contra el flujo real	Se corrigieron validaciones y estados cuando correspondía
Claude CLI - manual de usuario	Generar una base documental	Se revisó que los pasos coincidieran con el programa	Se adaptó el instructivo al funcionamiento final
Gemini - revisión BPMN	Consultar aspectos del modelado	Se contrastó con BPMN y la consigna	Se mantuvieron solo las propuestas coherentes con el proceso



Revisión de código y consignas a modo de verificación y corrección. Solicité a Claude que realice una auditoría del sistema buscando errores o inconsistencias a nivel código.

```
> Te paso una UML para que revises las consignas y me digas como voy en tema código y manual de usuario: file:///C:/Users/rwoyano/downloads/TPI%20OE-TUPad.pdf

• Voy a leer el PDF con las consignas.
  Searched for 1 pattern, read 1 file

• Voy a revisar todos los archivos del proyecto para tener el panorama completo antes de evaluarlo.
  Searched for 1 pattern

• Tengo el panorama completo. Acá te doy la evaluación punto por punto contra las consignas.

---
Estado actual del proyecto vs. consignas del TPI
Lo que ya tenés cubierto ✅



| Requisito                                    | Evidencia en tu proyecto                                                                                             |
|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Proceso administrativo identificado          | Toma de pedidos de Burger Home                                                                                       |
| Simulador funcional (Python)                 | chatbot_simulado.py                                                                                                  |
| Máquina de estados con "memoria"             | 6 estados: IDLE, ESPERANDO_PRODUCTO, ESPERANDO_MODALIDAD, ESPERANDO_DIRECCION, ESPERANDO_PAGO, ESPERANDO_COMPROBANTE |
| Persistencia con base de datos               | base_datos.json (lectura y escritura real)                                                                           |
| Al menos 2 compuertas (Gateways)             | Hay 4: ¿opción válida?, ¿hay stock?, ¿es Delivery?, ¿zona con cobertura?                                             |
| Robustez / Camino Infeliz                    | Opciones inválidas, sin stock, sin cobertura, comprobante muy corto                                                  |
| Diferenciación Tarea Usuario / Tarea Sistema | Está incluso comentado en el código con esos nombres exactos                                                         |
| Repositorio GitHub configurado               | Sí                                                                                                                   |
| Manual de Usuario                            | MANUAL_USUARIO.md recién creado                                                                                      |



---
Lo que te falta ❌

1. Diagrama BPMN 2.0 – Es el requisito más crítico y no existe ningún archivo de imagen o diagrama en el repo. Debe incluir:
  - Carriles (lanes): uno para "cliente" y uno para "Sistema/Bot"
  - Eventos de inicio y fin
  - Las 4 compuertas ya implementadas en el código
  - Flujo "as-is" (proceso manual actual) y "to-be" (proceso automatizado)

2. README.md – La defensa final exige específicamente un README.md que explique cómo desplegar el proyecto. El MANUAL_USUARIO.md que tenés cubre el uso, pero necesitás un README.md separado con instrucciones de instalación/ejecución.
  Jump to bottom (ctrl+end) 4
```

Consulte a Gemini a modo de revisión de los gráficos BPMN realizados y asegurarme de contemplar los aspectos técnicos de los diagramas BPMN 2.0



Revisa mis diagramas BPMN en busca de errores técnicos. Se me solicitó diagramar as-is y to-be para automatizar o hacer más eficiente un proceso administrativo

¡Buenas! Qué gran trabajo de mapeo armaste. Pasar de un proceso manual a uno automatizado manteniendo la coherencia en BPMN no es tarea fácil, pero conceptualmente capturaste

Aquí es donde la cátedra va a mirar con lupa la Perspectiva Técnica (que se traduzca en código, estados y persistencia). Hay varios errores de sintaxis BPMN críticos:

- **Tareas de Usuario vs. Tareas de Servicio/Script (Iconos en las tareas):**
 - Pusiste el ícono de User Task (el muñequito) en "Ingresar dirección de envío" y "Enviar comprobante". Esto está perfecto, porque lo hace el cliente.
 - Sin embargo, pusiste el ícono de Service Task / Script Task (los engranajes) en "Enviar menú disponible", "Solicitar modalidad de entrega", "Solicitar dirección", etc. ¡Ojo acá! El bot interactúa con el usuario. Cuando el bot envía un mensaje al cliente, en BPMN se modela como una Send Task (tarea de envío, ícono de un sobre negro) o como una User Task si se considera una interacción de interfaz de usuario. Los engranajes (Service Task) se reservan para backend puro (ej: "Validar zona de cobertura", "Calcular importe" o "Confirmar pedido y guardar en DB", los cuales sí están excelentemente clasificados).
- **Flujo roto en "Es delivery? = NO":** Si la compuerta "Es delivery?" va por el NO, la flecha salta directo al conector que está antes de "Calcular importe y solicitar pago". El problema es que se salta la compuerta de convergencia. En BPMN clásico, conectar un flujo directamente a una línea de secuencia sin usar una compuerta de unión para fusionarlos es un error de diseño.
- **Compuerta "En efectivo?":** * Si es NO (paga con tarjeta/transferecia), va a "Solicita comprobante".
 - Si es SÍ, la flecha va directamente hacia abajo, cruza la línea de la piscina y entra a "Recibir y preparar pedido" en la Cocina. ¡Cuidado! Te saltaste el nodo de convergencia del final. Además, significa que el cliente que paga en efectivo jamás recibe una confirmación de que su pedido fue tomado con éxito, ni el bot registra el estado en la base de datos. Faltaría que el flujo del SÍ se una más adelante para que también pase por "Confirmar pedido y guardar en DB".

- **Lógica del bucle del Comprobante Válido:** Al igual que en el *As-Is*, si el comprobante **NO** es válido, vas a *"Notifica y solicita nuevo comprobante"*, pero de ahí la flecha apunta directamente a *"Enviar comprobante"* (tarea del cliente). Conceptualmente se entiende, pero estás mezclando el carril del Chatbot con el del Cliente de forma abrupta a través de una tarea de usuario. Deberías hacer que la notificación del bot (carril Chatbot) active la recepción del nuevo comprobante.

MANUAL DE OPERACIÓN

Para ejecutar la simulación se requiere Python 3.10 o superior y que `chatbot_simulado.py` y `base_datos.json` se encuentren en el mismo directorio. Desde una terminal ubicada en la carpeta del proyecto se ejecuta: `python chatbot_simulado.py`.

El usuario inicia la conversación, selecciona un producto y define la modalidad de entrega. Si elige Delivery debe ingresar una dirección; si la zona no posee cobertura puede optar por Retiro. Luego selecciona el medio de pago y, cuando corresponde, ingresa el comprobante. Una vez confirmado, el sistema registra el pedido y actualiza el stock. Cocina informa la finalización de la preparación y el proceso continúa hasta que el pedido sea retirado o entregado. La palabra 'salir' permite finalizar la simulación de manera controlada.

REPOSITORIO Y CONCLUSIÓN

El proyecto se encuentra alojado en GitHub e incluye el script principal `chatbot_simulado.py`, el archivo `base_datos.json` utilizado para persistencia y el `README.md` con las instrucciones de ejecución. El código se encuentra organizado y comentado para facilitar su lectura y la demostración del flujo.

La propuesta permite relacionar el análisis organizacional con una implementación de software. El BPMN TO-BE representa las tareas y decisiones del proceso mejorado, mientras que la Máquina de Estados y las estructuras de decisión trasladan ese modelo a la simulación. La incorporación de caminos alternativos mejora la robustez frente a errores de entrada y evita finalizar pedidos que todavía pueden resolverse mediante una alternativa válida. De esta forma, Burger Home cuenta con una propuesta de automatización coherente entre el proceso de negocio, el modelo BPMN y la lógica programada.

DIAGRAMAS BPMN

Diagrama as-is:

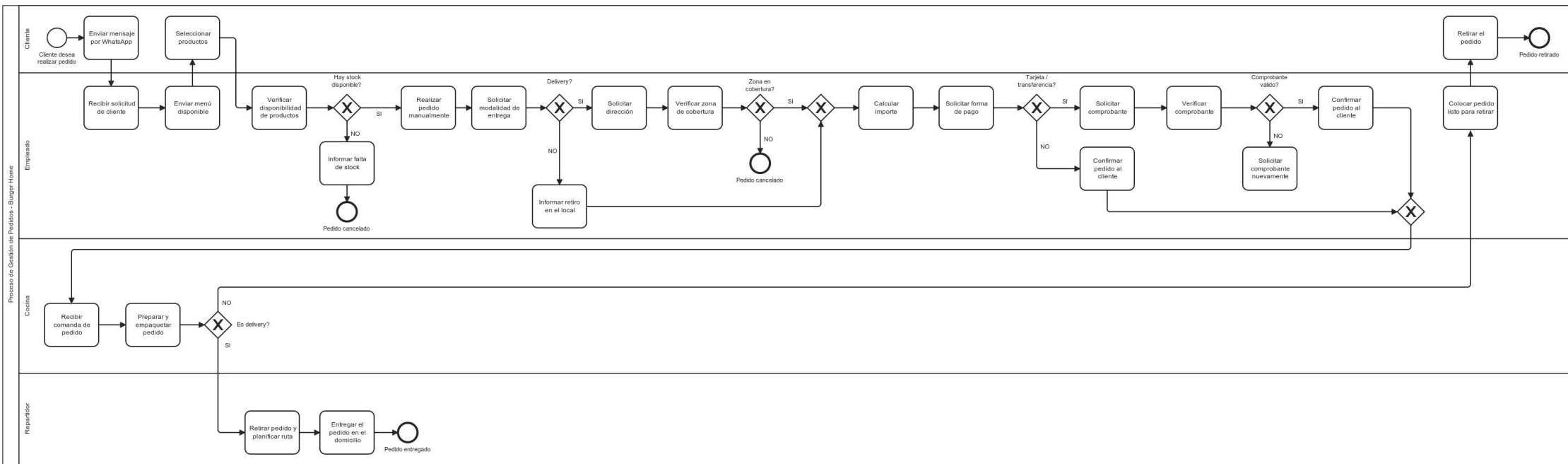


Diagrama to-be

